遊戲平台管理系統 - 系統設計文件（SD）參考用

> 版本：v8.1 旗艦版 + C# .NET 8 日期：2026 年 3 月 2 日 適用系統：遊戲平台管理系統 V8 (旗艦版)

---

1. 系統概述

1.1 系統目標

本系統旨在提供一個穩定可靠的遊戲平台管理解決方案，支援：

- 集中式後台統一管理
- 單機本地後台離線資料緩衝（遊戲需連線）
- 雙向資料同步與對帳
- 第三方遊戲 API 串接

1.2 系統邊界定義

集中式後台邊界

邊界元素	說明
管理範圍	所有機台、全平台遊戲、全部交易
資料擁有者	中央系統（唯一真實來源）
可用性要求	99.5% 正常運行時間
網路依賴	需持續連線（可離線一段時間）
職責	全域決策、帳務最終核對、系統設定

單機本地後台邊界

邊界元素	說明
管理範圍	單一機台、本機遊戲、本地交易
資料角色	本地緩衝（需連線集中式才能運作）

邊界元素	說明
可用性要求	離線獨立運作能力（網路恢復後同步）
網路依賴	可離線運作（網路恢復後與集中式同步）
職責	本地遊戲執行、本地交易紀錄、離線緩衝

> 重要：單機本地後台支援離線資料緩衝，但必須連線至集中式才能開始遊戲（保護帳務）。

邊界協定

協定類型	內容
通訊協定	HTTPS + WebSocket
資料交換格式	JSON
認證方式	JWT Token（中央）、PIN Code（本地）
同步時機	連線建立時自動同步，手動同步
衝突處理	中央優先（交易）、本地保留（日誌）

1.3 版本差異對照表

> 說明：本文件僅描述 旗艦版 **+ C# .NET 8** 之完整功能。

功能模組	旗艦版 **+ C# .NET 8**
中央後台	
- 機台管理	✅
- 交易管理	✅
- 系統使用者	✅
- 儀表板	✅ [L08]
- 即時機台監控	✅ [C79]
- 第三方遊戲商管理	✅ [C41]
- 代理商管理	✅
- 三層權限管理	✅
- 第三方錢包轉帳	✅
- OTA 遠端更新	✅
- 硬體監控	✅
- 錯帳對帳管理	✅

功能模組	旗艦版 + C# .NET 8
- 錢包自動對帳	✅
單機本地後台	
- PIN 登入	✅
- 開分/洗分	✅
- 離線交易	✅
- 同步機制	✅

1.4 系統架構

```
┌──────────────────────────────────────────────────────────────┐
│                          機台端應用程式                          │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                          單機本地後台                            │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐       │
│   │   Blazor     │   │ ASP.NET Core │   │   SQLite     │       │
│   │   Frontend   │   │   Backend    │   │  Database    │       │
│   └──────────────┘   └──────────────┘   └──────────────┘       │
└──────────────────────────────────────────────────────────────┘
                              │ HTTPS / WebSocket
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                          集中式後台                              │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐       │
│   │   Blazor     │   │ ASP.NET Core │   │ PostgreSQL   │       │
│   │   Frontend   │   │   Backend    │   │  Database    │       │
│   └──────────────┘   └──────────────┘   └──────────────┘       │
└──────────────────────────────────────────────────────────────┘
                              │
                   ┌──────────┴──────────┐
                   ▼                     ▼                     ▼
          ┌──────────────┐      ┌──────────────┐      ┌──────────────┐
          │  RG 電子     │      │  JDB 電子    │      │ 其他遊戲商   │
          └──────────────┘      └──────────────┘      └──────────────┘
```

1.5 閱讀指南

讀者	建議章節
專案經理	1, 7, 8, 9, 10
系統架構師	2, 3, 4, 11, 12

讀者	建議章節
開發工程師	3, 4, 5, 6, 16
維運工程師	11, 12, 13

1.6 快速開始

```
# 1. 複製專案
git clone game-platform.git

# 2. 安裝依賴
cd backend && npm install
cd ../frontend && npm install

# 3. 啟動開發環境
docker-compose up -d

# 4. 執行遷移
npm run migrate

# 5. 執行測試
npm test
```

---

2. 技術架構

2.1 前端技術

項目	技術	版本	備註
框架	Blazor	.NET 8	建議穩定版
UI 框架	Bootstrap 5	4.x	
路由	Blazor Router	latest	輕量路由庫
狀態管理	Blazor State	latest	推薦替代 Context
圖表	Chart.js	latest	
UI 元件	Radzen Blazor	latest	基於 Radzen

2.2 後端技術

項目	技術	版本	備註
執行環境	.NET 8	.NET 8 LTS	
框架	ASP.NET Core	4.x	

項目	技術	版本	備註
集中式資料庫	PostgreSQL	15.x	
本地資料庫	Microsoft.Data.Sqlite	3.x	原生效能更好
快取	Redis	7.x	
認證	jose	latest	JWT 處理庫
加密	AES-256-GCM + RSA	-	
WebSocket	SignalR	.NET 8	
API 文件	Swagger	-	

2.3 通訊架構

通訊方式	用途
RESTful API	資料 CRUD
WebSocket	即時狀態推送、心跳監控
HTTPS	安全傳輸 (TLS 1.3)

---

3. 資料庫設計

3.1 集中式資料庫（PostgreSQL）- 核心資料表

```sql
-- 機台表
CREATE TABLE machines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    location VARCHAR(200),
    machine_type VARCHAR(50),
    provider_id UUID REFERENCES providers(id),
    status VARCHAR(20) DEFAULT 'offline',
    heartbeat_interval INTEGER DEFAULT 60,
    last_heartbeat TIMESTAMP,
    software_version VARCHAR(20),
    hardware_info JSONB,
    config JSONB,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- 遊戲商表
CREATE TABLE providers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
```

```sql
    code VARCHAR(50) UNIQUE NOT NULL,
    api_endpoint VARCHAR(500),
    api_key_hash VARCHAR(255),
    api_secret_hash VARCHAR(255),
    webhook_url VARCHAR(500),
    status VARCHAR(20) DEFAULT 'active',
    balance DECIMAL(15,2) DEFAULT 0,
    min_balance_warning DECIMAL(15,2) DEFAULT 1000,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- 遊戲表
CREATE TABLE games (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id UUID REFERENCES providers(id),
    name VARCHAR(100) NOT NULL,
    game_code VARCHAR(50),
    category VARCHAR(50),
    status VARCHAR(20) DEFAULT 'active',
    rtp DECIMAL(5,2),
    thumbnail_url VARCHAR(500),
    created_at TIMESTAMP DEFAULT NOW()
);




-- 代理商表
CREATE TABLE agents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    parent_agent_id UUID REFERENCES agents(id),
    commission_rate DECIMAL(5,2) DEFAULT 0,
    balance DECIMAL(15,2) DEFAULT 0,
    status VARCHAR(20) DEFAULT 'active',
    created_at TIMESTAMP DEFAULT NOW()
);

-- 交易表
CREATE TABLE transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    machine_id UUID REFERENCES machines(id),
    game_id UUID REFERENCES games(id),
    provider_id UUID REFERENCES providers(id),
    operator_id VARCHAR(50),
    type VARCHAR(20) NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    balance_before DECIMAL(15,2),
    balance_after DECIMAL(15,2),
```

```sql
    external_tx_id VARCHAR(100),
    status VARCHAR(20) DEFAULT 'pending',
    error_code VARCHAR(50),
    created_at TIMESTAMP DEFAULT NOW()
);

-- 系統使用者表
CREATE TABLE system_users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL,
    agent_id UUID REFERENCES agents(id),
    permissions JSONB DEFAULT '[]',
    last_login_at TIMESTAMP,
    failed_login_attempts INTEGER DEFAULT 0,
    locked_until TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 稽核日誌表
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID,
    user_type VARCHAR(20),
    action VARCHAR(100) NOT NULL,
    resource_type VARCHAR(50),
    resource_id UUID,
    old_value JSONB,
    new_value JSONB,
    ip_address VARCHAR(45),
    created_at TIMESTAMP DEFAULT NOW()
);
```

3.2 本地資料庫（SQLite）

```sql
-- 本地交易
CREATE TABLE local_transactions (
    id TEXT PRIMARY KEY,
    game_id TEXT,
    type TEXT NOT NULL,
    amount REAL NOT NULL,
    balance_before REAL,
    balance_after REAL,
    status TEXT DEFAULT 'pending',
    sync_attempts INTEGER DEFAULT 0,
    created_at TEXT DEFAULT (datetime('now'))
);

-- 同步佇列
```

```sql
CREATE TABLE sync_queue (
    id TEXT PRIMARY KEY,
    data TEXT NOT NULL,
    retry_count INTEGER DEFAULT 0,
    max_retries INTEGER DEFAULT 5,
    status TEXT DEFAULT 'pending',
    error_message TEXT,
    created_at TEXT DEFAULT (datetime('now'))
);

-- 本機設定
CREATE TABLE local_settings (
    key TEXT PRIMARY KEY,
    value TEXT,
    updated_at TEXT DEFAULT (datetime('now'))
);

-- PIN 認證
CREATE TABLE local_auth (
    pin_hash TEXT NOT NULL,
    pin_salt TEXT NOT NULL,
    failed_attempts INTEGER DEFAULT 0,
    locked_until TEXT
);
```

3.3 索引設計

```sql
-- 交易表索引
CREATE INDEX idx_transactions_machine ON transactions(machine_id);
CREATE INDEX idx_transactions_created ON transactions(created_at
        DESC);
CREATE INDEX idx_machines_status ON machines(status);
CREATE INDEX idx_audit_logs_user ON audit_logs(user_id);
CREATE INDEX idx_audit_logs_created ON audit_logs(created_at DESC);
```

---

4. API 設計

4.1 API 回應格式

```jsonc
// 成功
{ "success": true, "data": {}, "pagination": {} }

// 失敗
{ "success": false, "error": { "code": "AUTH_001", "message": "" } }
```

4.2 API 端點 (精簡版)

認證 [Auth]

- POST /api/v1/auth/login
- POST /api/v1/auth/logout
- GET /api/v1/auth/verify
- POST /api/v1/auth/refresh

機台 [C01-C35]

- GET/POST /api/v1/machines
- GET/PUT/DELETE /api/v1/machines/:id
- POST /api/v1/machines/:id/command

遊戲商 [C41]

- GET/POST /api/v1/providers
- GET/PUT /api/v1/providers/:id
- GET /api/v1/providers/:id/balance

錢包 [C106]

- GET /api/v1/wallet/balances
- POST /api/v1/wallet/transfer

交易 [C36/C37]

- GET /api/v1/transactions
- POST /api/v1/transactions/:id/reverse

OTA [C77]

- GET/POST /api/v1/ota/versions
- POST /api/v1/ota/deploy

爭議處理 [L20]

- GET /api/v1/disputes
- POST /api/v1/disputes
- PATCH /api/v1/disputes/:id

錯帳 [L20]

- GET /api/v1/discrepancies
- POST /api/v1/discrepancies/:id/resolve

4.3 API 錯誤碼

錯誤碼	說明	HTTP
AUTH_001	登入失敗	401
AUTH_002	Token 過期	401
AUTH_003	權限不足	403
AUTH_004	帳號已停用	403
RES_001	資源不存在	404
VAL_001	參數驗證失敗	400
VAL_002	餘額不足	400
TXN_001	交易失敗	500
TXN_002	交易處理中	202
SYNC_001	同步失敗	500
EXT_001	第三方 API 錯誤	502

4.4 API 結構總覽

本系統 API 共 **35** 個端點，分為 **11** 個群組。 請參照 API_v1_openapi.yaml

API 端點統計表

群組	端點數	HTTP 方法
認證 (Auth)	4	POST, GET
機台管理 (Machines)	6	GET, POST, PUT, DELETE
錢包管理 (Wallet)	2	GET, POST
遊戲管理 (Games)	5	GET, POST, PUT, DELETE

交易管理 (Transactions) | 2 | GET, POST |
遊戲商管理 (Providers) | 3 | GET, POST |
多人遊戲 (Multiplayer) | 4 | GET, POST, PUT |
代理商管理 (Agents) | 2 | GET, POST |
OTA 更新 (OTA) | 3 | GET, POST |
監控中心 (Monitor) | 1 | GET |
爭議處理 (Dispute) | 3 | GET, POST, PATCH |
系統備份 (System) | 2 | POST |
錯帳管理 (Discrepancies) | 2 | GET, POST |
同步管理 (Sync) | 2 | GET, POST |
總計 | 36 | |

---

5. 離線同步機制

5.1 同步原則

1. **最終一致性**: 允許短暫不一致，最終達成一致
2. **中央優先**: 衝突時全數以中央設定為準（強制覆蓋本地狀態）
3. **冪等性**: 支援重複同步不會造成問題
4. **可恢復性**: 失敗可重試，不丟失資料
5. **遠端複寫**: 集中管理可對單機本地後台覆寫設定（需本地在線，記錄操作日誌）
6. **本地即時同步**: 本地每次修改立刻同步至集中端
7. **斷線補齊**: 離線連線後立即執行同步

5.2 重試策略 (指數退避)

重試次數	延遲時間
1	30 秒
2	1 分鐘
3	2 分鐘
4	5 分鐘
5	10 分鐘
6+	15 分鐘 (固定)

超過最大重試次數:

- 標記為失敗
- 產生告警
- 進入待處理人工處理

5.3 同步流程

```
本地後台                                          集中式後台
    |                                                 |
    |---- POST /api/v1/sync/upload ----------------->|
    |     { transactions: [...], version: x }         |
    |                                                 |
    |<---- 200 OK ----------------------------------|
    |     { sync_version, server_time }               |
    |                                                 |
    |---- GET /api/v1/sync/download ---------------->|
    |     Query: since_version=x                      |
    |                                                 |
    |<---- 200 OK ----------------------------------|
```

```
│     { changes: [...], version: y }          │
│                                             │
│──── 處理下載資料 ──────────────────────────►│
│     • 衝突檢測                              │
│     • 資料合併                              │
│                                             │
│◄──── 200 OK ────────────────────────────────│
│     { applied: true, conflicts: [] }        │
│                                             │
│──── 確認同步完成 ──────────────────────────►│
│     Body: { last_sync: y }                  │
│                                             │
│◄──── 200 OK ────────────────────────────────│
│     Body: { success: true }                 │
```

5.4 衝突解決策略

本系統採用**分層衝突解決策略**：

自動解決（系統層面）

衝突類型	解決方式	優先順序	說明
重複交易	中央檢查 UUID 去重	中央優先	防止重複派彩
版本衝突	本地優先	本地優先	以本地送上之config為主（已apply）
設定衝突	中央優先	中央優先	本地修改立即同步至集中
遠端覆寫	遠端優先	遠端優先	集中端主動覆寫時以遠端為準

人工介入（爭議專區）

衝突類型	解決方式	優先順序
餘額衝突	保留本地記錄，進入爭議專區	人工裁決
帳務異常	保留本地記錄，進入爭議專區	人工裁決

> ⚠️ **核心原則**：所有帳務相關衝突（餘額、交易）絕不自動刪除本地記錄，必須進入「爭議對帳處理專區」由人工處理。

5.5 遠端設定複寫流程

當集中管理需要對單機本地後台進行遠端設定複寫時，遵循以下流程：

```
集中式後台                                          單機本地後台
    |                                                   |
    |  1. 管理員發起遠端設定請求                          |
    |                                                   |
    |---- POST /api/v1/config/remote-push ------------->|
    |     { config_type, config_value, version,         |
    |       request_id, operator_id }                   |
    |                                                   |
    |                                                   |  2. 驗證本地在線狀
態
    |                                                   |
    |<---- 200 OK (本地在線) ----------------------------|
    |     { status: "online", can_apply: true }         |
    |                                                   |
    |                                                   |  3. 套用設定到本地
端
    |                                                   |  4. 記錄操作日誌
    |                                                   |
    |---- POST /api/v1/config/apply-result ------------>|
    |     { request_id, applied: true/false,            |
    |       error_msg?, applied_at }                    |
    |                                                   |
    |  5. 儲存結果至集中DB                                |
    |  6. 記錄操作日誌（操作者、時間、結果）               |
    |                                                   |
    |<---- 200 OK ------------------------------------- |
    |     { success: true, stored_at }                  |
```

遠端複寫 API 規格

API	方法	說明
`/api/v1/config/remote-push`	POST	發起遠端設定複寫
`/api/v1/config/apply-result`	POST	回傳套用結果
`/api/v1/config/sync`	POST	本地主動同步設定

遠端複寫限制

- 單機本地後台需在線才能接受遠端複寫
- 每次複寫需記錄完整操作日誌
- 失敗需回傳錯誤訊息供集中端記錄
- 遊戲供應商、串接遊戲、多人遊戲不支援遠端複寫

5.6 離線同步邊界處理

```
// 離線期間交易處理
const offlineHandler = {
  // 1. 本地交易隊列
  queue: [],

  // 2. 離線期間餘額計算
  calculateBalance: (localTxs, serverBalance) => {
    const localDelta = localTxs
      .filter(tx => tx.status === 'pending')
      .reduce((sum, tx) => {
        return tx.type === 'deposit'
          ? sum + tx.amount
          : sum - tx.amount;
      }, 0);

    return {
      available: serverBalance - localDelta,
      pending: localDelta
    };
  },

  // 3. 衝突解決
  resolveConflict: (localTx, serverTx) => {
    if (localTx.status === 'synced') {
      return serverTx; // 中央版本優先
    }

    // 本地有待同步交易
    if (serverTx.status === 'failed') {
      return { ...localTx, status: 'retry' };
    }

    return serverTx;
  }
};
```

---

6. 安全性設計

6.1 認證與授權

JWT Token 設計

- **Access Token**: 15 分鐘有效期
- **Refresh Token**: 7 天有效期
- **Token 輪換**: 自動刷新机制

動態 RBAC 角色權限

本系統採用**動態角色自定義引擎**，支援業主手動勾選高達上百個細部 API 權限，指派給自定義角色。

角色範本	說明	適用對象
審計員	可查看所有交易、報表，無操作權限	財務稽核
查帳會計	可查看交易歷史與帳務報表	会计人员
維修外包	僅有機台操作權限，無帳務權限	設備維護
站點管理	單一站點的所有權限	店长

PIN 碼保護

- 供單機本地端快速登入使用，與 `system_users` 綁定
- 4-6 位數字
- bcrypt 雜湊儲存（對應 `system_users.pin_hash`）
- 透過「使用者管理」功能（系統管理權限）進行初設與重置
- 錯誤 5 次後鎖定 30 分鐘

6.2 權限模型 - 動態自定義 RBAC

本系統支援動態自定義權限，詳見 6.1 節所述。

6.3 權限繼承與細部權限

權限類別	細項權限
機台管理	機台列表檢視、機台狀態監控、遠端開分/洗分、遠端重啟、機台配置下發
機台帳務	交易列表檢視、交易詳情檢視、機台餘額調整
交易管理	交易列表檢視、交易詳情檢視、交易統計、異常交易處理
系統管理	系統設定、使用者管理、角色管理、權限管理

6.3 資料加密

項目	演算法	用途
密碼儲存	bcrypt (cost: 12)	密碼雜湊

項目	演算法	用途
敏感資料	AES-256-GCM	欄位加密
傳輸加密	TLS 1.3	HTTPS
訊息驗證	HMAC-SHA256	資料完整性
API 簽名	RSA-2048	遊戲商 API
PIN 儲存	bcrypt + salt	本地 PIN

6.4 安全 Headers

```
const securityHeaders = {
  'X-Content-Type-Options': 'nosniff',
  'X-Frame-Options': 'DENY',
  'X-XSS-Protection': '1; mode=block',
  'Strict-Transport-Security': 'max-age=31536000; includeSubDomains',
  'Content-Security-Policy': "default-src 'self'",
};
```

6.4 API 安全

```
// API 請求驗證中間件
const apiSecurity = {
  // 1. 請求 ID 追蹤 (防止重放攻擊)
  requestId: {
    generate: () => crypto.randomUUID(),
    validate: (req, res, next) => {
      const requestId = req.headers['x-request-id'];
      // 檢查請求 ID 是否已使用 (Redis 記錄)
      // 過期時間: 5 分鐘
    }
  },

  // 2. API 簽名驗證
  signature: {
    algorithm: 'HMAC-SHA256',
    validate: (req, body, secret) => {
      const signature = req.headers['x-api-signature'];
      const expected = crypto
        .createHmac('sha256', secret)
        .update(JSON.stringify(body))
        .digest('hex');
      return signature === expected;
    }
  },

  // 3. Rate Limiting
  rateLimit: {
    // 分散式 Rate Limit 使用 Redis
```

```
    store: new RedisStore({
      prefix: 'rl:',
      sendCommand: (...args) => redis.call(...args)
    })
  }
};
```

6.5 交易安全

```
// 交易處理最佳實踐
class TransactionService {
  async createTransaction(machineId, amount, type) {
    // 1. 樂觀鎖定
    const machine = await db.machines.findById(machineId);
    if (machine.version !== req.headers['if-match']) {
      throw new AppError('VAL_005', '資料已更新，請重新整理');
    }

    // 2. 使用資料庫交易
    return await db.transaction(async (trx) => {
      // 扣款
      const updated = await trx('machines')
        .where('id', machineId)
        .where('balance', '>=', amount)
        .decrement('balance', amount);

      if (updated === 0) {
        throw new AppError('VAL_002', '餘額不足');
      }

      // 記錄交易
      const tx = await trx('transactions').insert({...});

      return tx;
    });
  }
}
```

6.6 程式碼安全防護

SQL 注入防護

```
-- 使用參數化查詢
-- ❌ 錯誤範例
SELECT * FROM machines WHERE name = '${machineName}'

-- ✅ 正確範例
SELECT * FROM machines WHERE name = $1
```

並發處理 (樂觀鎖定)

```
// 樂觀鎖定範例
const optimisticLock = {
  // 使用 version 欄位
  updateMachine: async (machineId, updates) => {
    const machine = await db.machines.findById(machineId);

    const result = await db.machines
      .where('id', machineId)
      .where('version', machine.version)
      .update({ ...updates, version: machine.version + 1 });

    if (result === 0) {
      throw new AppError('VAL_005', '並發衝突，請重試');
    }

    return true;
  }
};
```

---

7. 第三方遊戲商串接

7.1 支援遊戲商

遊戲商	代碼	遊戲類型	API 版本
RG 電子	rg	老虎機、捕魚	v2
JDB 電子	jdb	老虎機、棋牌	v2
RT 電子	rt	老虎機	v2

7.2 串接流程

```
1. 新增遊戲商設定
   ├── API Endpoint
   ├── API Key
   ├── API Secret
   └── Webhook URL

2. 測試連線
   ├── API /health
   └── 餘額查詢

3. 正式上線
   ├── 遊戲開通
```

```
    ├── 錢包建立
    └── 交易對接
```

7.3 錢包轉帳流程

```
機台 ──▶ 中央錢包
           │
           ▼
     ── 第三方 API ──
     │   (轉入遊戲)
     │
     ▼
遊戲商錢包 ──▶ 遊戲中
                │
                ▼
           ── 第三方 API ──
           │   (轉出中央)
           │
           ▼
      中央錢包 ──▶ 機台
```

7.4 第三方 API 超時處理

```
// 第三方 API 調用超時處理
const externalApiHandler = {
  timeout: 10000, // 10 秒

  callWithRetry: async (fn, maxRetries = 3) => {
    for (let i = 0; i < maxRetries; i++) {
      try {
        return await Promise.race([
          fn(),
          new Promise((_, reject) =>
            setTimeout(() => reject(new Error('TIMEOUT')),
        this.timeout)
          )
        ]);
      } catch (err) {
        if (i === maxRetries - 1) throw err;
        await sleep(Math.pow(2, i) * 1000); // 指數退避
      }
    }
  }
};
```

---

8. OTA 遠端更新

8.1 更新流程

```
1. 上傳新版本
   ├── 檔案上傳 (最大 500MB)
   ├── SHA-256 驗證
   └── 版本資訊

2. 測試派發
   ├── 選擇 2-3 台機台
   ├── 觀察 24 小時
   └── 確認無問題

3. 正式派發
   ├── 全部派發 / 分批派發
   ├── 即時 / 預約
   └── 強制更新 / 選擇更新

4. 監控進度
   ├── 即時進度
   ├── 失敗重試
   └── 異常告警
```

8.2 版本相容性

- 只支援從舊版本升級
- 重大版本需強制更新
- 支援差異更新 (Delta)
- 版本號格式: x.y.z (major.minor.patch)

---

9. 硬體監控

9.1 監控項目

項目	警告閾值	嚴重閾值
CPU 負載	> 80%	> 90%
記憶體使用	> 80%	> 90%
硬碟空間	< 15%	< 5%
網路延遲	> 100ms	> 500ms
溫度	> 60°C	> 75°C

9.2 心跳機制

- 機台每分鐘發送心跳
- 超過 3 分鐘無心跳 → 離線
- 離線告警通知管理員

9.3 告警動作

嚴重度	通知方式
警告	Telegram
嚴重	Telegram + SMS + 電話

---

10. 錯帳對帳管理

10.1 錯帳類型

類型	說明
missing_deposit	轉帳未到帳
missing_withdrawal	轉出未確認
payout_error	派彩錯誤
system_error	系統錯誤

10.2 處理流程

```
1. 偵測錯帳
   ├── 自動偵測（對帳發現）
   └── 人工回報（客戶投訴）

2. 調查原因
   ├── 檢查日誌
   ├── 聯繫遊戲商
   └── 調取交易紀錄

3. 處理方式
   ├── 補發點數
   ├── 退款
   └── 沖銷

4. 產生憑證
   ├── PDF 憑證
```

```
    ├── 寄送財務
    └── 歸檔
```

10.3 錢包對帳

- 每日自動對帳 (UTC 00:00)
- 發現差異立即告警
- 支援人工對帳
- 差異報告匯出

10.4 爭議對帳處理專區 [L20]

功能定義：用於處理機台交易異常、或系統與遊戲商之間的交易爭議，包含建立爭議單、調查、裁決與補救。

爭議單建立

1. 選擇爭議類型 (如：deposit_not_arrived, payout_error)。
2. 關聯原始交易編號。
3. 填寫問題描述與期望處理方式。

調查與裁決

- **狀態追蹤**：待處理 -> 調查中 -> 裁決中 -> 結案。
- **補救措施**：餘額調整、補發交易、沖銷交易、派彩修正。

補救操作邏輯

```
async function executeRemedy(disputeId, authCode) {
  // 1. 驗證主管授權碼
  // 2. 根據裁決結果執行對應 API (如 /machines/:id/balance)
  // 3. 更新爭議單狀態為 closed
  // 4. 寫入稽核日誌
}
```

---

11. 營收報表功能

11.1 功能概述

營收報表模組提供全平台的多維度營收分析能力，支援多條件交叉篩選與圖表化呈現。

11.2 篩選維度

維度	類型	說明
門市/分店	多選	按門市進行營收篩選
機台	多選	按機台進行營收篩選
遊戲	多選	按遊戲進行營收篩選
遊戲商	多選	按遊戲供應商進行營收篩選

11.3 報表類型

類型	週期	用途
日報表	每日	每日營收統計
週報表	每週	每週營收統計
月報表	每月	每月營收統計
年報表	每年	每年營收統計

11.4 營收欄位

欄位	說明
總營收	派彩 - 下注
下注總額	總下注金額
派彩總額	總派彩金額
遊戲局數	遊戲總回合數
營收(RTP)	派彩率

11.5 API 端點

```yaml
/reports/revenue:
  GET:
    summary: 營收報表
    parameters:
      - startDate: 開始日期
      - endDate: 結束日期
      - reportType: 報表類型 (daily/weekly/monthly/yearly)
      - stores: 門市篩選 (多選)
      - machines: 機台篩選 (多選)
      - games: 遊戲篩選 (多選)
      - providers: 遊戲商篩選 (多選)

/reports/revenue/export:
```

```yaml
  GET:
    summary: 匯出營收報表
    parameters:
      - format: 匯出格式 (csv/excel)
```

11.6 查詢優化

- 使用 PostgreSQL 聚合函數進行高效計算
- 建立複合索引優化多維度篩選效能
- Redis 快取常用报表数据

---

12. 部署架構

11.1 生產環境架構

```
                    ┌──────────────┐
                    │    CDN       │
                    │ (靜態資源)   │
                    └──────────────┘
                          │
                          ▼
                    ┌──────────────┐
                    │   Nginx      │
                    │  (Reverse    │
                    │   Proxy)     │
                    └──────────────┘
                          │
              ┌───────────┴───────────┐
              ▼                       ▼
        ┌──────────┐            ┌──────────┐
        │ .NET 8 │            │ .NET 8 │
        │ Backend 1│            │ Backend 2│
        └──────────┘            └──────────┘
              │                       │
              └───────────┬───────────┘
                          │
              ┌───────────┴───────────┐
              ▼                       ▼
        ┌──────────┐            ┌──────────┐
        │ PostgreSQL │          │   Redis  │
        │  Primary   │          │   Cache  │
        └──────────┘            └──────────┘
              │
              ▼
        ┌──────────┐
        │ PostgreSQL │
        │  Read      │
```

```
        │  Replica   │
        └────────────┘
```

11.2 容器化部署

```yaml
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DB_HOST=postgres
      - REDIS_HOST=redis
    depends_on:
      - postgres
      - redis
    restart: unless-stopped

  postgres:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=game_platform
      - POSTGRES_USER=admin
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    volumes:
      - redisdata:/data
    command: redis-server --appendonly yes
    restart: unless-stopped

volumes:
  pgdata:
  redisdata:
```

---

12. 效能設計

12.1 效能目標

項目	目標
API 響應時間 (P95)	< 200ms
資料庫查詢 (P95)	< 500ms
WebSocket 延遲	< 100ms
系統可用性	≥ 99.5%
支援機台數	1000+
支援同時在線機台	500+

12.2 優化策略

策略	實作
資料庫索引	針對常用查詢建立複合索引
Redis 快取	熱門資料快取 5 分鐘
CDN	靜態資源快取
非同步處理	交易紀錄寫入隊列
連線池	資料庫連線池管理
查詢優化	避免 SELECT *，使用分頁

12.3 效能優化實踐

資料庫連線池

```
// PostgreSQL 連線池優化
const pool = new Pool({
  max: 30,              // 最大連線數 (根據 CPU 核心數調整)
  min: 10,              // 最小連線數
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 5000,
  // 洪水保護
  maxUses: 10000,       // 連線最大使用次數
  allowExitOnIdle: false
});

// 監控連線池健康
pool.on('error', (err) => {
  logger.error('Pool error:', err);
});
```

Redis 快取策略

```
// 快取策略
const cacheStrategy = {
  // 1. 熱門資料 (短期快取)
  hotData: {
    ttl: 300,           // 5 分鐘
    prefix: 'hot:',
    keys: ['dashboard:*', 'machines:online:*']
  },

  // 2. 配置資料 (中期快取)
  configData: {
    ttl: 3600,          // 1 小時
    prefix: 'cfg:',
    keys: ['config:*', 'providers:*']
  },

  // 3. 使用者會話 (短期)
  sessionData: {
    ttl: 900,           // 15 分鐘
    prefix: 'sess:',
    keys: ['session:*']
  }
};
```

N+1 查詢優化

```
// ❌ N+1 查詢
const machines = await db.machines.findAll();
for (const machine of machines) {
  const provider = await db.providers.findById(machine.provider_id);
  machine.provider = provider;
}

// ✅ 使用 JOIN 或 eager loading
const machines = await db.machines.findAll({
  include: ['provider']
});

// ✅ 或使用 GraphQL-style selection
const machines = await db.machines
  .select('machines.*', 'providers.name as provider_name')
  .join('providers', 'machines.provider_id', 'providers.id');
```

---

13. 監控與日誌設計

13.1 監控項目

類別	項目
機台監控	心跳、狀態、連線
API 監控	響應時間、錯誤率
資料庫	連線池、查詢效能
同步	成功率、延遲
資源	CPU、記憶體、磁碟

13.2 告警規則

條件	嚴重度	動作
機台離線 > 15 分鐘	警告	Telegram
機台離線 > 60 分鐘	嚴重	Telegram + SMS
API 錯誤率 > 5%	警告	Telegram
API 錯誤率 > 20%	嚴重	Telegram + 電話
同步失敗 > 3 次	警告	Telegram
硬碟空間 < 10%	警告	Telegram
錯帳金額 > $1000	警告	Telegram + Email

13.3 日誌與系統追蹤

結構化日誌

```
// JSON 結構化日誌
const logger = {
  info: (message, meta) => {
    console.log(JSON.stringify({
      timestamp: new Date().toISOString(),
      level: 'info',
      message,
      ...meta,
      service: 'game-platform',
      version: '1.0.0'
    }));
  },

  error: (message, meta) => {
    console.error(JSON.stringify({
```

```
      timestamp: new Date().toISOString(),
      level: 'error',
      message,
      ...meta,
      stack: new Error().stack,
      service: 'game-platform',
      version: '1.0.0'
    }));
  }
};
```

請求追蹤

```
// 請求 ID 傳遞
const requestTracking = {
  // 產生追蹤 ID
  generate: (req, res, next) => {
    req.id = req.headers['x-request-id'] || crypto.randomUUID();
    res.setHeader('x-request-id', req.id);
    next();
  },

  // 記錄請求日誌
  log: (req, res, next) => {
    const start = Date.now();

    res.on('finish', () => {
      logger.info('HTTP Request', {
        method: req.method,
        path: req.path,
        status: res.statusCode,
        duration: Date.now() - start,
        requestId: req.id
      });
    });

    next();
  }
};
```

---

14. 錯誤處理機制

14.1 全域錯誤處理

```
// 錯誤分類
class AppError extends Error {
  constructor(code, message, statusCode = 500) {
    super(message);
```

```
    this.code = code;
    this.statusCode = statusCode;
  }
}

// 錯誤處理中介軟體
app.use((err, req, res, next) => {
  logger.error({
    error: err.message,
    stack: err.stack,
    requestId: req.id
  });

  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      success: false,
      error: {
        code: err.code,
        message: err.message
      }
    });
  }

  // 未預期錯誤
  return res.status(500).json({
    success: false,
    error: {
      code: 'SYS_001',
      message: '系統錯誤'
    }
  });
});
```

14.2 交易錯誤處理流程

```
交易請求
    │
    ▼
驗證參數 ──► 錯誤 ──► 返回 VAL_001
    │
    ▼
檢查餘額 ──► 不足 ──► 返回 VAL_002
    │
    ▼
開啟交易 ──► 失敗 ──► 重試 3 次
    │
    ▼
記錄結果 ──► 錯誤 ──► 標記pending + 告警
    │
```

▼
返回結果

---

15. 資料遷移策略

15.1 升級遷移

```
-- 遷移腳本

-- 新增遊戲商欄位
ALTER TABLE machines ADD COLUMN provider_id UUID;

-- 新增代理商相關欄位
ALTER TABLE system_users ADD COLUMN agent_id UUID;
ALTER TABLE system_users ADD COLUMN permissions JSONB DEFAULT '[]';

-- 建立新表
CREATE TABLE providers (...);
CREATE TABLE agents (...);

-- 建立索引
CREATE INDEX idx_machines_provider ON machines(provider_id);
CREATE INDEX idx_agents_parent ON agents(parent_agent_id);
```

15.2 資料備份策略

類型	頻率	保留期
完整備份	每日 (UTC 02:00)	30 天
增量備份	每小時	7 天
交易備份	即時 (WAL)	1 年
設定備份	每週	90 天

---

16. 開發與測試規範

16.1 測試策略

單元測試

```
// 交易服務單元測試
describe('TransactionService', () => {
  it('should deduct balance on bet', async () => {
```

```
    const machine = await createTestMachine({ balance: 1000 });

    await transactionService.createTransaction({
      machineId: machine.id,
      amount: 100,
      type: 'bet'
    });

    const updated = await getMachine(machine.id);
    expect(updated.balance).toBe(900);
  });

  it('should throw error on insufficient balance', async () => {
    const machine = await createTestMachine({ balance: 50 });

    await expect(
      transactionService.createTransaction({
        machineId: machine.id,
        amount: 100,
        type: 'bet'
      })
    ).rejects.toThrow('VAL_002');
  });
});
```

整合測試

```
// API 整合測試
describe('POST /api/v1/transactions', () => {
  it('should create transaction successfully', async () => {
    const response = await request(app)
      .post('/api/v1/transactions')
      .set('Authorization', `Bearer ${token}`)
      .send({
        machineId: machineId,
        amount: 100,
        type: 'deposit'
      });

    expect(response.status).toBe(201);
    expect(response.body.success).toBe(true);
  });
});
```

16.1.1 遊戲測試功能技術設計

本系統提供遊戲測試功能，允許管理員在正式環境中進行虛擬下注以驗證遊戲邏輯。

功能描述

項目	說明
功能名稱	遊戲測試 - 虛擬下注 (Game Test)
適用對象	系統管理員、維修人員
使用情境	現場機台驗證、遊戲邏輯測試
API 設計	根據遊戲供應商 API 動態設計

虛擬下注流程

```
1. 啟動測試模式
   ├── 驗證管理員權限
   ├── 建立測試會話 (Session)
   └── 記錄測試開始時間

2. 執行虛擬下注
   ├── 模擬下注請求 (不扣除真實餘額)
   ├── 記錄虛擬交易紀錄
   └── 顯示遊戲結果

3. 測試結果記錄
   ├── 記錄測試日誌
   ├── 產生測試報告
   └── 結束測試會話
```

API 端點設計

API	方法	說明
`/api/v1/games/:id/test/start`	POST	啟動遊戲測試模式
`/api/v1/games/:id/test/bet`	POST	執行虛擬下注
`/api/v1/games/:id/test/end`	POST	結束測試模式
`/api/v1/games/:id/test/report`	GET	取得測試報告

資料表設計

```sql
-- 遊戲測試記錄表
CREATE TABLE game_test_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    game_id UUID REFERENCES games(id),
    machine_id UUID REFERENCES machines(id),
    tester_id UUID REFERENCES system_users(id),
    test_session_id VARCHAR(100) NOT NULL,
    bet_amount DECIMAL(15,2),
    win_amount DECIMAL(15,2),
```

```
    game_result JSONB,
    test_status VARCHAR(20) DEFAULT 'in_progress',
    started_at TIMESTAMP DEFAULT NOW(),
    ended_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);
```

權限控制

- 僅有 系統管理員 和 維修外包 角色可以使用遊戲測試功能
- 測試模式需要在啟動時記錄操作者 ID
- 測試期間的所有操作都會記錄到稽核日誌

虛擬交易格式

```
// 虛擬交易請求格式
const virtualBetRequest = {
  machineId: "test-machine-id",
  gameId: "game-uuid",
  amount: 100,
  isVirtual: true,  // 虛擬下注標記
  testSessionId: "session-uuid"
};

// 虛擬交易響應
const virtualBetResponse = {
  success: true,
  data: {
    betId: "virtual-bet-uuid",
    isVirtual: true,
    result: {
      win: true,
      winAmount: 150,
      multiplier: 1.5
    },
    testSessionId: "session-uuid"
  }
};
```

16.2 格式與編碼標準化

程式碼區塊

```
// 統一使用 TypeScript 類型標註
interface Transaction {
  id: string;
  machineId: string;
  amount: number;
  type: 'bet' | 'payout' | 'deposit' | 'withdraw';
```

```
  status: 'pending' | 'success' | 'failed';
  createdAt: Date;
}
```

SQL 命名規範

```sql
-- 表名: 蛇形命名 (snake_case)
CREATE TABLE machine_transactions (...);

-- 欄位名: 蛇形命名
machine_id, created_at, is_active

-- 索引名: idx_{表名}_{欄位名}
idx_machines_status
```

API 命名規範

```
# RESTful 規範
# 資源: 名詞 (複數)
/api/v1/machines
/api/v1/transactions

# 動作: HTTP 方法
GET    # 查詢
POST   # 新增
PUT    # 更新 (完整)
PATCH  # 更新 (部分)
DELETE # 刪除

# 子資源
/api/v1/machines/{id}/transactions
/api/v1/machines/{id}/heartbeat
```

---

17. 版本發布與支援

17.1 版本發布流程

發布檢查

```
1. 文件審查
   ├── ✅ 所有章節完成
   ├── ✅ 程式碼範例驗證
   └── ✅ 交叉引用正確

2. 技術審查
   ├── ✅ API 設計合理
```

```
    ├── ✅ 安全性檢查通過
    └── ✅ 效能考量充分

3. 格式檢查
    ├── ✅ Markdown 格式正確
    ├── ✅ 表格對齊
    └── ✅ 圖表清晰

4. 發布
    ├── ✅ Git commit
    ├── ✅ Tag 建立
    └── ✅ 版本號更新
```

版本號規範

```
v{major}.{minor}.{patch}

major: 架構變更 (不相容)
minor: 新功能 (向後相容)
patch: 錯誤修復

範例:
v1.0.0 - 初版
v1.1.0 - 新增 OTA 功能
v1.1.1 - 修復 API 錯誤
v2.0.0 - 架構重構
```

17.2 聯絡與支援

問題回報格式

```
**環境**
- 版本:
- 部署方式:

**問題描述**
[詳細說明問題]

**重現步驟**
1.
2.

**預期結果**
[說明預期]

**實際結果**
[說明實際]
```

日誌

[附上相關日誌]

版本	日期	說明	作者
v1.0	2026-02-25	初版系統設計	docs
v1.1	2026-02-26	新增版本差異、API 錯誤碼、第三方串接、OTA、硬體監控、錯帳管理章節	docs
v1.2	2026-02-26	技術審查修訂：API 版本化、錯誤碼擴充、資料庫優化	code
v1.3	2026-02-26	擴充章節：同步、安全、部署、監控、遷移	docs+code
v1.4	2026-02-26	深度技術審查 + 最終審查 (Round 4-5)	code+docs

文件維護：工程師 審查狀態：✅ Round 4-5 完成 (v1.4)